

Test cases for Binary Search Tree:

Test Case 1: Insertion of Contacts

Description: Insert multiple contacts into the BST.

Input:

```
bst.insert("Tushar", "123-456-7890")
```

```
bst.insert("Amol", "234-567-8901")
```

```
bst.insert("Zakir", "345-678-9012")
```

```
bst.insert("Khushi", "456-789-0123")
```

```
bst.insert("Pushpak", "567-890-1234")
```

Expected Result:

- The contacts should be inserted in such a way that names are ordered lexicographically in an in-order traversal.
- No errors should occur.

Observation:

- The insertion worked as expected. All contacts were inserted into the BST, maintaining the lexicographical order.
-

Test Case 2: Searching for Existing Contacts

Description: Search for contacts that have already been inserted into the BST.

Input:

```
print("Search for Tushar:", bst.search("Tushar"))
```

```
print("Search for Amol:", bst.search("Khushi"))
```

Expected Result:

- The search for "Tushar" should return "123-456-7890".
- The search for "Khushi" should return "456-789-0123".

Observation:

- The correct phone numbers were returned for both "Tushar" and "Khushi". The search function traversed the tree correctly to find the names.
-

Test Case 3: Searching for a Non-existent Contact

Description: Search for a contact that does not exist in the BST.

Input:

```
print("Search for Rahul:", bst.search("Rahul"))
```

Expected Result:

- The search for "Rahul" should return None, as the contact does not exist in the BST.

Observation:

- The search correctly returned None for the non-existent contact "Rahul".
-

Test Case 4: In-order Traversal (Print Contacts)

Description: Perform an in-order traversal of the BST to print all contacts in lexicographical order.

Input:

```
bst.inorder_traversal()
```

Expected Result:

- The names should be printed in lexicographically sorted order:

Amol: 234-567-8901

Khushi: 456-789-0123

Pushpak: 567-890-1234

Tushar: 123-456-7890

Zakir: 345-678-9012

Observation:

- The in-order traversal worked as expected, printing the names and phone numbers in the correct order.

Test Case 5: Deleting a Contact (Leaf Node)

Description: Delete a contact ("Pushpak") and perform an in-order traversal to verify the deletion.

Input:

```
bst.delete("Pushpak")  
print("\nAfter Deleting Pushpak:")  
bst.inorder_traversal()
```

Expected Result:

- The contact "Pushpak" should be removed from the BST.
- The updated in-order traversal should print:

Amol: 234-567-8901

Khushi: 456-789-0123

Tushar: 123-456-7890

Zakir: 345-678-9012

Observation:

- The contact "Pushpak" was successfully deleted, and the in-order traversal no longer included "Pushpak". The BST structure was correctly updated.

Test Case 6: Deleting a Contact (Node with One Child)

Description: Delete a contact ("Khushi") that has one child and verify the in-order traversal afterward.

Input:

```
bst.delete("Khushi")  
print("\nAfter Deleting Khushi:")  
bst.inorder_traversal()
```

Expected Result:

- The contact "Khushi" should be removed, and its single child should take its place in the BST.
- The updated in-order traversal should print:

Amol: 234-567-8901

Pushpak: 567-890-1234

Tushar: 123-456-7890

Zakir: 345-678-9012

Observation:

- The deletion of "Khushi" worked as expected. The node with one child was correctly removed, and its child was moved to the correct position.

Test Case 7: Deleting a Contact (Node with Two Children)

Description: Delete a contact ("Tushar") that has two children and verify the in-order traversal afterward.

Input:

```
bst.delete("Tushar")
```

```
print("\nAfter Deleting Tushar:")
```

```
bst.inorder_traversal()
```

Expected Result:

- The contact "Tushar" should be removed, and the in-order successor (smallest node in the right subtree) should replace it.
- The updated in-order traversal should print:

Amol: 234-567-8901

Khushi: 456-789-0123

Pushpak: 567-890-1234

Zakir: 345-678-9012

Observation:

- The deletion of "Tushar" correctly replaced the deleted node with the in-order successor. The BST maintained its structure, and the in-order traversal produced the expected results.
-

Test Case 8: Deleting a Non-existent Contact

Description: Attempt to delete a contact that does not exist in the BST.

Input:

```
bst.delete("Rahul")  
print("\nAfter Deleting Rahul:")  
bst.inorder_traversal()
```

Expected Result:

- The BST should remain unchanged, as "Rahul" is not present.

Amol: 234-567-8901

Khushi: 456-789-0123

Pushpak: 567-890-1234

Tushar: 123-456-7890

Zakir: 345-678-9012

Observation:

- The tree remained unchanged after attempting to delete the non-existent contact "Rahul". No errors occurred.